Weeks 7 to 11: Lecture and Lab Plan

Each week's topic, how it connects to the final project, the 20-minute lab walkthrough and the one-hour lab at home

How each week works

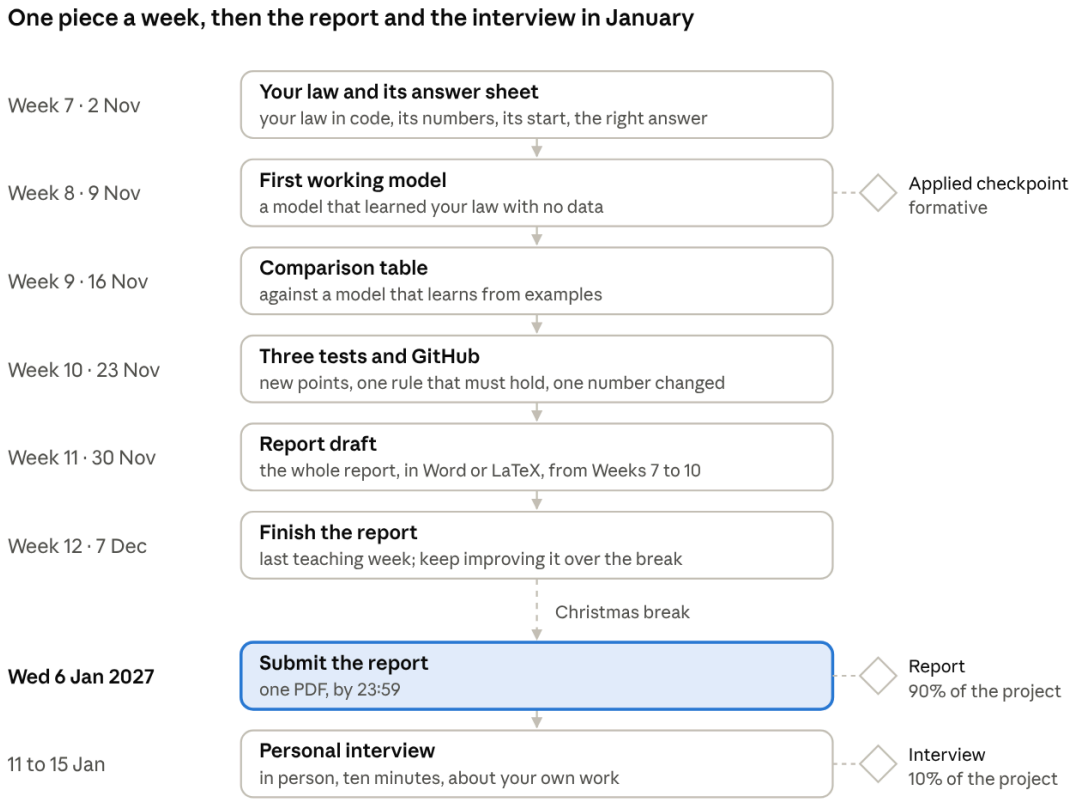
Every week has one recorded 2-hour online lecture and one 1-hour lab at home. The lecture teaches the week's AI topic properly, then explains how that topic connects to the project, then walks through the project steps for 20 minutes. At home, each student does the same steps on their own project. All work is individual, and the project is built step by step, one piece a week.

Part	Time	What happens
The topic	60 min	The week's AI topic, taught in full, in plain words.
How it connects to your project	30 min	What the topic means for the project, and the project ideas needed this week.
Lab walkthrough	20 min	The lecturer goes through this week's project steps on the class example.
Wrap-up	10 min	What to hand in this week. Questions go to the channel.
Lab at home	1 hour	Part A (15 min): a small hands-on task on the topic. Part B (45 min): the project steps on their own law, then a short form.

The class example: a sourdough starter rising in a jar. It grows fast, then slows down as it nears the top of the jar. It is not one of the twelve project ideas, so the demo never does a student's project for them.

The project is the one from the idea form: a model that learns from a law of science instead of data, checked against the right answer worked out another way, and compared with a model that learns from examples.

The road to the final project



the road to the final project · Week 7 (2 November 2026) to the January interviews

Each week adds one piece. By the end of Week 11, students have a full draft of their final report.

Week 7 (from 2 November 2026) · Large language models

By the end of the week: students understand how large language models work and where they are hitting limits, and have their own law written down with a correct answer to check against.

The topic (60 min)

- **What a large language model (LLM) does.** It reads some text and guesses the most likely next word, then the next, one word at a time. Answering questions, writing code and summarising all come from that one skill, done extremely well.
- **What is inside (a reminder from Week 4).** A transformer. Its attention step lets every word look at every other word to decide what matters, which is why LLMs follow long sentences so well.
- **How an LLM is made, in three stages.** First, pretraining: it reads a huge library of text and practises guessing the next word, for weeks, on thousands of computers. Second, fine-tuning: it practises one job, such as answering questions helpfully. Third, learning from feedback (RLHF): people rate its answers, and it learns to give the kind people prefer.
- **Tokens.** Before the model reads text, the text is cut into pieces called tokens: whole words, parts of words, punctuation. Cost and speed are counted in tokens. Tokens also explain some odd mistakes, such as miscounting the letters in a word.

- **Emergent abilities.** Some skills seem to appear only once models are big enough. Researchers disagree about whether they really appear suddenly, or only look sudden because of how we measure them.
- **Open and closed models.** Open models (Llama, DeepSeek) can be downloaded and run on your own computer, so your data stays with you. Closed models (ChatGPT) are used online; they are often more capable, but you pay, and your data goes to the company.
- **Scaling, and where it strains.** For years, a bigger model with more data and more computing power meant a better model. Now each step up gives a smaller gain, and the costs are huge: money, electricity, and running out of good new text to learn from. That is why Weeks 8 and 9 look at faster and smaller models.

How it connects to your project (30 min)

- **The opposite approach.** An LLM learns everything from data: billions of examples. Your project learns from a law of science, with no data at all. When data is scarce or expensive (a new machine, a rare illness), a law can do what data cannot.
- **Why it matters now.** The limits of scaling are exactly why small models that know a law are interesting: they are tiny and cheap by comparison.
- **Use an LLM as a helper, then check it.** An LLM can explain your law to you, but it can be confidently wrong. Check what it says against the student guide; it is Week 5's habit again: check, don't trust.
- **This week's project step: write your law so a computer can use it.**
  - The law as one line of code. Move everything to one side, so that it equals zero when the law is obeyed (the "leftover" from Week 5).
  - Its numbers (constants), each with a unit and where you found it.
  - Its starting value, and the values at both ends if your law changes along a line.
  - The answer sheet: the right answer worked out another way, with an exact formula or a solver, so you can test your model later.
  - Keep the numbers near 1 (hours rather than seconds); models learn much better that way.

Lab walkthrough (20 min): the sourdough starter

1. Open capstone_step1.ipynb.
2. Write the law as one line: dHdt - r * H * (1 - H / K).
3. Set the numbers: r = 0.5 per hour, the top of the jar K = 12 cm, starting height 2 cm, from 0 to 16 hours.
4. Make the answer sheet with the exact formula and with the solver, and plot both; they should lie on top of each other.
5. Show the YOUR LAW cells where students put their own law.

Lab at home (1 hour)

- **Part A (15 min):** run a small model on your laptop with Ollama. Put one sentence about your project into the Tokeniser Explorer and count the tokens. Ask the local model and ChatGPT to explain your law, and note any mistakes.
- **Part B (45 min):** follow the walkthrough steps with your own law. Hand in: your law as one line of code, your numbers and where you found them, your starting value, and a picture of your answer sheet.

Week 8 (from 9 November) · Faster language models

By the end of the week: students understand why long text is slow and expensive for language models and what the newer designs do about it, and have a first model that has learned their own law.

The topic (60 min)

- **Why long text is slow.** In attention, every word is compared with every other word. 1,000 words means about a million comparisons; 2,000 words means about four million. Twice the text, four times the work and memory.
- **State-space models (S4, Mamba, Mamba-2).** They read the words in order and keep a small running summary, like a notepad, updated at each word. Twice the text means only twice the work, and the memory stays the same size. It is the Week 4 reader with a notepad, trained in a smarter way.
- **Other attention-free designs (RetNet, RWKV).** Different recipes with the same aim: cost that grows only in step with the length of the text.
- **Mixture of experts.** The model is split into many "experts". For each word, a small router picks just a few of them to do the work. You get a very large model that costs about as much as a small one to run.
- **Reasoning models.** Instead of making the model bigger, let it think step by step for longer before it answers. Each answer costs more, but harder questions get solved.
- **The lesson to keep.** Always measure cost (time and memory) as well as accuracy.

How it connects to your project (30 min)

- **The same lesson in your project.** From now on, record your model's cost as well as its accuracy: training time and size. Week 9's comparison needs both.
- **This week's project step: train a model that learns your law, as in Week 5.**
  - How it learns: guess, check the guess against the law at many points in time (the leftover should be zero), work out the blame, nudge. No data is used anywhere.
  - The starting value is built in with a fade-in, so it is always right.
  - If your law has two or three quantities, the model has one output for each. If your law changes along a line, the model has two inputs: place and time.
  - Decide in advance what "good enough" means, as a number (your success test), so you cannot move the goalposts later.
  - Read the loss curve: falling then flattening is good; stuck high means it has not found the law yet; jumping about means the steps are too big.
  - If it fails, try four fixes in order: keep the numbers near 1; use more checking points; use a shorter time range; take smaller steps.

Lab walkthrough (20 min): the sourdough starter

1. Open capstone_step2.ipynb with the sourdough law, numbers and start already in.
2. Train it (about two minutes) and read the loss curve.
3. Plot the model on top of the answer sheet and read the error.
4. Check the success test: "the height at 8 hours is within 0.1 cm".
5. Break it on purpose (only 10 checking points), show the failure, and fix it.

Lab at home (1 hour)

- **Part A (15 min):** in the speed visualiser, compare the normal model and the newer model on 1,000 and 32,000 tokens of text. Which is faster for long text, and why?
- **Part B (45 min):** follow the walkthrough steps with your own law. Hand in: your error, your success test and whether it passed, training time, model size, a picture of the model against the answer sheet, and one problem you fixed.

Applied checkpoint (formative, feeds the final project): students add a short report on where this first model could go wrong: what the law leaves out, the numbers they chose, and Week 5's wrong-law lesson. It is not separately graded; it becomes the limits section of the final project's brief, and it is evidence for the project's process mark.

Week 9 (from 16 November) · Smaller language models

By the end of the week: students know the main ways to make a model smaller and why it matters, and have compared their own model with one that learns from examples.

The topic (60 min)

- **Why small matters.** A small model is cheaper to run, keeps data private on your own device, uses less energy, works without the internet, and more people can afford to use it.
- **Distillation.** A big "teacher" model answers many questions, and a small "student" model learns to copy its answers. The student is far smaller but keeps much of the skill. A transformer can even teach a faster, recurrent-style model this way.
- **Quantization.** Every number inside a model is normally stored with 16 or 32 binary digits. Storing them with 8, 4 or even 1 or 2 makes the model 4 to 16 times smaller, usually with only a small drop in quality.
- **Pruning.** Remove the connections that barely matter, like trimming a hedge.
- **LoRA and QLoRA.** To teach a model a new job, keep it frozen and train a small add-on instead of every number. QLoRA does this on a quantized model, so it fits on a laptop.
- **Small models in the real world.** Small language models run on phones; tiny models (TinyML) run on sensors; energy use matters at this scale; neuromorphic chips, which work more like brain cells, use very little power.

How it connects to your project (30 min)

- **Your model is already small.** A model that learns a law has a few thousand numbers, not billions. This week you show it, with numbers.
- **This week's project step: the comparison the final project needs.**
  - Race your model against an ordinary model that learns from examples: points taken from your answer sheet, with a little noise added, like real measurements.
  - Keep it fair: the same size and the same training time; only what it learns from is different.
  - Measure three things: accuracy (error against the answer sheet), speed (time per answer), and size (number of numbers, file size).
  - Answer the question you chose on the idea form: can no data match examples; how many examples the ordinary model needs to catch up; which is smaller and faster; which still works outside the range it learned on.
  - Then apply today's topic to your own model: shrink it (fewer numbers, fewer digits) until it fails your success test, and keep the last version that passed.

Lab walkthrough (20 min): the sourdough starter

1. Open capstone_step3.ipynb with the sourdough example in.
2. Train the ordinary model on 4, 8, 16, 32 and 64 examples.
3. Read the table (accuracy, speed, size) and the chart of error against number of examples.
4. Run the outside-the-range test.

5. Shrink the sourdough model, and say the finding in one sentence with a number.

Lab at home (1 hour)

- **Part A (15 min):** with Ollama, run a normal and a quantized (shrunk) version of the same small model. Note the size and speed of each. Which would you put on a phone?
- **Part B (45 min):** follow the walkthrough steps with your own law. Hand in: your table, your chart, and your finding in one sentence with a number.

Week 10 (from 23 November) · AI for building software

By the end of the week: students understand how AI is used to build software and systems, and have tested their own model honestly and put it on GitHub.

The topic (60 min)

- **AI coding assistants.** Tools such as GitHub Copilot, Claude Code and ChatGPT suggest code, explain errors and write tests. They are good at routine code, but they can invent functions that do not exist and still sound sure. You must read and understand every line you hand in.
- **Agents.** An agent is an AI that carries out several steps on its own: write code, run it, read the result, fix it, repeat. It is faster, but it needs someone checking what it does.
- **Retrieval-augmented generation (RAG).** Before answering, the system searches your own documents for the most relevant passages and hands them to the model, like an open-book exam. Answers are less often made up, and they can point to their sources.
- **Embeddings and vector databases.** Each passage is turned into a list of numbers that captures its meaning. Passages with similar meanings get similar numbers, so the system can quickly find the ones that match a question.
- **Designing a system.** Use a small model on your own computer for private, cheap, quick jobs, and a large online model for the hard ones.
- **Augmentation, not replacement.** AI makes developers faster; it does not replace understanding. Always say how you used AI.
- **Good habits for any project.** Keep it on GitHub, write a README, fix the random seed so results repeat, and make it run with one command.

How it connects to your project (30 min)

- **You may use an AI assistant this week,** to help write your tests and your README, as long as you check every line and say how you used it.
- **This week's project step: three honest tests, from Week 5's warnings.**
  - **New points.** A small error only covers the points you checked. Test your model at points it never saw.
  - **Something that must always be true.** Ask three questions of your law: what can never happen? What must stay the same? What can only go one way? Turn one answer into a test.
  - **The wrong-law test.** Change one number in your law by 20%. How far does the answer move? Would your model's own score have warned you?
  - Write one sentence on what your law leaves out: your model cannot know it, so you must say it.
  - Optional: find an unknown number in your law from a few noisy readings.
  - Put everything on GitHub, with a README.

Lab walkthrough (20 min): the sourdough starter

1. Open capstone_step4.ipynb with the sourdough example in.
2. Test at new points between the ones it was checked at.
3. Test what must always be true: the starter never rises above the top of the jar, and never shrinks.
4. Change r by 20% and see what happens.
5. Find r from five noisy readings, and compare with the true 0.5.
6. Put it on GitHub and fill in one README section.

Lab at home (1 hour)

- **Part A (15 min):** ask the booklet notebook (a small RAG system over the Week 5 booklet) two questions, and check one answer against the booklet page it points to.
- **Part B (45 min):** follow the walkthrough steps with your own law. Hand in: your GitHub link, the result of each test, your sentence on what the law leaves out, and how you used AI.

Week 11 (from 30 November) · What is next in AI

By the end of the week: students can describe where AI is heading and tell real progress from hype, and have a full draft of their final report.

The topic (60 min)

- **AI as a tool for science.** AlphaFold predicts the 3D shape of proteins, and AlphaFold 3 extends this to how molecules fit together, which helps drug design. GNoME proposed hundreds of thousands of possible new materials. AI now designs proteins that stick to chosen targets. Weather and climate models made with AI produce forecasts far faster than traditional ones.
- **The critical view.** These predictions still have to be tested in a lab, and many proposed materials cannot actually be made. AI speeds up experiments; it does not replace them.
- **AI at the edge.** More AI runs directly on phones, cars and sensors: faster, more private, and working without the internet.
- **Federated learning.** Many phones improve one shared model together. Each phone learns from its own data and sends back only the improvement, never the data itself; phone keyboards learn this way.
- **Other frontiers, briefly.** Neuromorphic and spiking chips that work more like brain cells; quantum machine learning, still very early; AI agents that act on their own; world models that learn how the physical world behaves.
- **Real progress or hype?** Test any AI headline with Week 1's seven questions.

How it connects to your project (30 min)

- **Your project is part of this trend.** A small model that knows a law of science is where science-inspired AI and small, lean AI meet. Your report can say so, with an example from today.
- **This week's project step: a full draft of the final report.**
  - One detailed report holds everything (90% of the project mark): the person and the problem; the law; the answer sheet; the model; the comparison; the checks and limits; how a large language model would tackle the same job; how AI tools shaped your own way of working; whether science-inspired AI (and edge, federated or quantum approaches) is real progress or hype for your person; and appendices with the repository link and the AI-use log. Students write it in Word or LaTeX, using the sample report outline and templates.

- Most of the report already exists by now: Weeks 7 to 10 produced its figures, tables and numbers. This week is writing it up.
- The interview (10%), in person in the week of 11 January 2027, checks that the work is the student's own: they explain their law line, their comparison, one entry in their AI-use log, what a 20% wrong law would mean, and how an LLM would tackle the problem.

Lab walkthrough (20 min): the sourdough starter

1. Walk through the sample report outline section by section, showing what goes in each one for the sourdough starter (for a home baker deciding when the starter is ready). It shows the shape of a report only; each student's content is their own.
2. Show the Word template and the LaTeX template (in Overleaf), and where the figures and tables from Weeks 7 to 10 go.
3. Export the sourdough report as a PDF, named StudentNumber_Surname_Report.pdf.

Lab at home (1 hour)

- **Part A (15 min):** find one news headline about AI in science. Is it real progress or hype? Give two reasons, using Week 1's seven questions.
- **Part B (45 min):** write a full draft of your report in Word or LaTeX, using the outline and a template. Hand in: your report draft as a PDF, and the one section you are least sure about.

Week 12: the last teaching week (from 7 December 2026), with no lecture topic: students work on their report, and can keep improving it over the Christmas break. After the break, the report is due as one PDF by 23:59 on Wednesday 6 January 2027, and each student has a 10-minute personal interview, in person, with Dr Indrakshi Dey in the week of Monday 11 January 2027. The report is 90% of the project mark and the interview 10%. There is no showcase.

Your weekly forms

Each lab at home ends with a short form on Tutors (about ten questions), due before the next lecture:

Form	Due	What you hand in
Week 7	before the Week 8 lecture (9 November)	your law, numbers, starting value, answer-sheet picture
Week 8	before the Week 9 lecture (16 November)	error, pass or fail, model picture, one problem fixed
Week 9	before the Week 10 lecture (23 November)	table, chart, one finding
Week 10	before the Week 11 lecture (30 November)	GitHub link, three test results, how you used AI
Week 11	before Week 12 (7 December)	report draft (PDF), the section you are least sure about